

Lab session
Advanced Computerarchitecture

(B-KUL-JLI1QS)

Renaut Mestdagh

February 16, 2024



Class: MEIsa

Instructor: Alexander Marinšek

Contents

1 Report **2**

2 Code **4**

1 Report

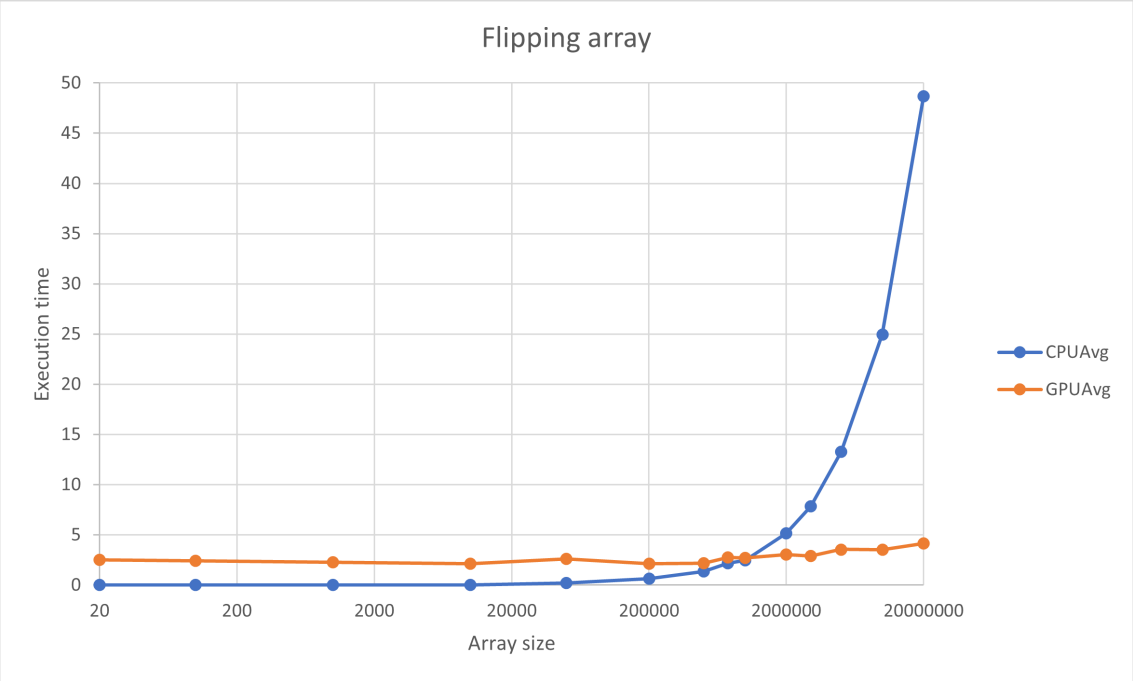
This lab is an intro into GPU programming. The GPU programming will happen by the CUDA API. CUDA is a software layer that gives direct access to the GPU's virtual instruction set and parallel computational elements for the execution of compute kernels. CUDA is designed to work with programming languages such as C, C++, and Fortran.[1] We will use C++.

The exercise in this lab is to flip an array. This must be done in parallel so we must use CUDA. I programmed and timed it when executing on CPU and GPU. This way we can really see the difference.

First of all, we create the to flip array. The size of the array doesn't really matter but the bigger the array, the better the GPU will perform in comparisson with the CPU. The values in the array also don't matter. They can be any value. Next we allocate the size of the array two times on the GPU. Once for the input array and once for the output value. After that we copy the input array from the CPU memory to the GPU memory. Now we have to decide how many threads each block should have and how many blocks we want. Each block should have between 1 and 1024 threads. Since all threads of a block are expected to reside on the same streaming multiprocessor core and must share the limited memory resources of that core, there is a limit to the number of threads per block. However, a kernel can be executed by multiple equally-shaped thread blocks, so that the total number of threads is equal to the number of threads per block times the number of blocks.[2] I chose to put 1024 threads in a block. The calculate the number of blocks, we use the following formula. $blocks = \frac{arrSize+threadsPerBlock-1}{threadsPerBlock}$. This way, each element from the array has thread. There is a (high) possibility there are too many threads. This doesn't hurt much. Only their creation will take some unnecessary time. Now we can start flipping the array. Fist of all, each thread calculates the index of the element of the array it should handle. Then all threads place its element in their correct position in the output array. Once all threads are done, the output array gets copied from the GPU memory to the CPU memory. Now we can deallocate the memory we allocated on the GPU. The array is flipped.

To see the speed difference for this operation, I did the same exersice but on the CPU. I timed the GPU and CPU execution times. For each array length, I ran it 3 times and calculated the average running time for CPU and GPU. This way we can eliminate some inconsistencies. We can see that in the beginning, the CPU is a lot faster. At around an array size of 1.000.000, the GPU execution time dips down the CPU execution time. From then on, the CPU execution time rises quickly while the GPU execution time rises slightly.

[1] https://en.wikipedia.org/wiki/CUDA
[2] https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#thread-hierarchy



Figuur 1: CPU vs GPU

2 Code

```
#include <iostream>
#include <cuda.h>
#include <cuda_runtime.h>
#include <ctime>

__global__ void flip_array(const int* input, int* output, int size) {
    unsigned int index = threadIdx.x + blockIdx.x * blockDim.x; // Thread ID
    if (index < size)
        output[size - index - 1] = input[index]; // Access and flip element
}

int main() {
    // Array size
    const int size = 50000000;

    // Allocate host memory
    int* input = new int[size];
    int* outputCPU = new int[size];

    // Initialize input array
    for (int i = 0; i < size; ++i) {
        input[i] = i * 2; // Example values
    }

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Allocate device memory
    int* d_input, *d_output;
    cudaMalloc(&d_input, size * sizeof(int));
    cudaMalloc(&d_output, size * sizeof(int));

    // Copy input to device
    cudaMemcpy(d_input, input, size * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel with multiple blocks and threads
    const int threadsPerBlock = 1024;   // max 1024
    const int blocks = (size + threadsPerBlock - 1) / threadsPerBlock;
    cudaEventRecord(start);
    flip_array<<<blocks, threadsPerBlock>>>(d_input, d_output, size);
    cudaEventRecord(stop);

    // Copy output back to host
    cudaMemcpy(input, d_output, size * sizeof(int), cudaMemcpyDeviceToHost);

    // Free memory
    cudaFree(d_input);
    cudaFree(d_output);
```

```
    cudaEventSynchronize(stop);
    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    std::cout << "GPU time:" << milliseconds << std::endl;

//    for (int i = 0; i < size; i++)
//        std::cout << input[i] << " ";
//    std::cout << std::endl;

    cudaEventRecord(start);
    for (int i = 0; i < size; i++)
        outputCPU[i] = input[size-i-1];
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);
    milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);
    std::cout << "CPU time:" <<milliseconds << std::endl;

//    for (int i = 0; i < size; i++)
//        std::cout << outputCPU[i] << " ";
//    std::cout << std::endl;

    delete[] input;
    delete[] outputCPU;

    return 0;
}
```